Theory of Computation

Lecture 5: Regular Operations and Regular Expressions

Sheikh Azizul Hakim
Lecturer, Department of CSE, BUET

Last compiled: June 28, 2026

We have machines that compute.

Now we need an algebra to describe what they compute.

Today's Roadmap

This lecture follows the central arc of Sipser's treatment of regular languages.

1. Define the three **regular operations**.
2. Prove regular languages are closed under them.
3. Define **regular expressions** formally.
4. Prove that regular expressions and finite automata have exactly the same power.

The goal is not just to memorize constructions, but to see one idea repeat:

$$\text{language operation} \quad \Longleftrightarrow \quad \text{machine construction.}$$

Languages as Sets

Recall that a **language** is simply a set of strings.

Because languages are sets, we can combine them using set operations to build new, more complex languages.

In automata theory, three specific operations are fundamental. We call these the **regular operations**.

Alphabet, Strings, and Languages

Fix an alphabet Σ.

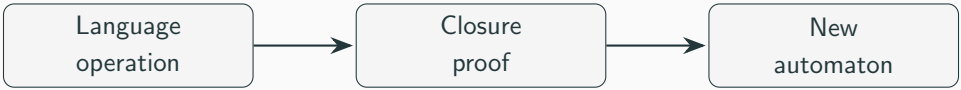
Object	Meaning
Σ	a finite set of symbols
w	a finite string over Σ
ε	the empty string
Σ^*	the set of all strings over Σ
$A \subseteq \Sigma^*$	a language over Σ

A regular operation takes one or more languages and returns another language.

Operations on Languages Are Not Operations on Machines

When we write $A \cup B$, AB, or A^*, we are talking about sets of strings.

The machine comes later.



Closure proofs answer the question: if the inputs have machines, can we build a machine for the output language?

The Three Regular Operations

Let A and B be languages. We define the regular operations as follows:

1. **Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
2. **Concatenation:** $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
3. **Star:** $A^* = \{x_1 x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$

Understanding Union

Definition: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$

The union simply takes all strings from A and all strings from B and puts them into a single set.

Example: Let $\Sigma = \{a, b\}$.

$$A = \{\text{good}, \text{bad}\}$$
$$B = \{\text{boy}, \text{girl}\}$$

$$A \cup B = \{\text{good}, \text{bad}, \text{boy}, \text{girl}\}$$

Understanding Concatenation

Definition: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Concatenation attaches a string from A to the front of a string from B in all possible combinations.

Example: Let $A = \{\text{good}, \text{bad}\}$ and $B = \{\text{boy}, \text{girl}\}$.

$$A \circ B = \{\text{goodboy}, \text{goodgirl}, \text{badboy}, \text{badgirl}\}$$

Understanding Concatenation

Definition: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Concatenation attaches a string from A to the front of a string from B in all possible combinations.

Example: Let $A = \{\text{good}, \text{bad}\}$ and $B = \{\text{boy}, \text{girl}\}$.

$$A \circ B = \{\text{goodboy}, \text{goodgirl}, \text{badboy}, \text{badgirl}\}$$

Note: We often omit the circle and just write AB.

Understanding Kleene Star

Definition: $A^* = \{x_1 x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$

The star operation is a **unary** operation. It attaches any number of strings in A together to get a new string.

Because k can be 0, the empty string ε is **always** in A^*, regardless of what A is.

Example: Let $A = \{ab, c\}$.

$$A^* = \{\varepsilon, ab, c, abab, abc, cab, cc, ababab, ababc, \dots\}$$

Why Star Always Contains ε

For a language A, the expression A^* means concatenate **zero or more** strings from A.

If we concatenate zero strings, the result is the empty string:

$$A^* = \{\varepsilon\} \cup A \cup AA \cup AAA \cup \cdots$$

So:

$$\varepsilon \in A^* \quad \text{for every language } A.$$

Why Star Always Contains ε

For a language A, the expression A^* means concatenate **zero or more** strings from A.

If we concatenate zero strings, the result is the empty string:

$$A^* = \{\varepsilon\} \cup A \cup AA \cup AAA \cup \cdots$$

So:

$$\varepsilon \in A^* \quad \text{for every language } A.$$

This is why $\emptyset^* = \{\varepsilon\}$, not $\emptyset$.

Regular Languages

Definition: A language is called a **regular language** if some DFA recognizes it.

Alternative Definition: A language is called a **regular language** if some NFA recognizes it.

Both are valid since a DFA is trivially an NFA and every NFA has an equivalent DFA.

Regular Languages

Definition: A language is called a **regular language** if some DFA recognizes it.

Alternative Definition: A language is called a **regular language** if some NFA recognizes it.

Both are valid since a DFA is trivially an NFA and every NFA has an equivalent DFA.

The Big Question: If we take two regular languages and apply a regular operation to them, is the resulting language still regular?

Sipser's Closure Theorem

Sipser proves the following theorem early because it becomes the bridge to regular expressions.

Theorem. The class of regular languages is closed under the regular operations:

$$\cup, \qquad \circ, \qquad {}^*.$$

Sipser's Closure Theorem

Sipser proves the following theorem early because it becomes the bridge to regular expressions.

Theorem. The class of regular languages is closed under the regular operations:

$$\cup, \qquad \circ, \qquad {}^*.$$

The proof uses NFAs because ε-transitions make the constructions visually and formally simple.

This is also why nondeterminism is not a side topic; it is the cleanest language-design tool we have.

What is a Closure Property?

A collection of objects is **closed** under some operation if applying that operation to members of the collection returns an object still in the collection.

Mathematical Examples:

- The set of natural numbers $\mathbb{N}$ is closed under addition.
  $(3 + 5 = 8 \in \mathbb{N})$
- $\mathbb{N}$ is **not** closed under subtraction.
  $(3 - 5 = -2 \notin \mathbb{N})$

Closure of Regular Languages

Theorem: The class of regular languages is closed under the union operation.

In other words, if A_1 and A_2 are regular languages, then $A_1 \cup A_2$ is also a regular language.

Closure of Regular Languages

Theorem: The class of regular languages is closed under the union operation.

In other words, if A_1 and A_2 are regular languages, then $A_1 \cup A_2$ is also a regular language.

Proof Idea: Since A_1 and A_2 are regular, there exist machines M_1 and M_2 that recognize them. We must build a new machine M that recognizes $A_1 \cup A_2$.

Building the Union Machine

How does M accept an input w? It should accept if either M_1 accepts w or M_2 accepts w.

We *could* build a DFA that simulates M_1 and M_2 simultaneously (keeping track of pairs of states).

Building the Union Machine

How does M accept an input w? It should accept if either M_1 accepts w or M_2 accepts w.

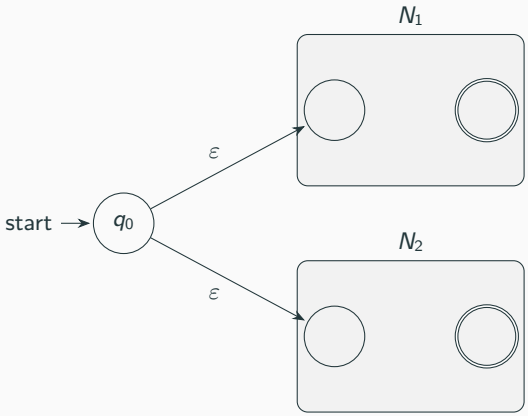
We *could* build a DFA that simulates M_1 and M_2 simultaneously (keeping track of pairs of states).

But remember our new tool: **Nondeterminism**.

With an NFA, we can just *guess* which machine will accept!

Proof: Closure under Union

Let N_1 recognize A_1 and N_2 recognize A_2. We construct a new NFA N to recognize $A_1 \cup A_2$.



Add a new start state q_0 and ε-transitions to the start states of N_1 and N_2.

Union Construction: Formal Version

Let

$$N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1), \qquad N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$$

recognize A_1 and A_2.

Construct $N = (Q, \Sigma, \delta, q_0, F)$ where:

$$Q = Q_1 \cup Q_2 \cup \{q_0\},$$
$$F = F_1 \cup F_2.$$

The new transition function keeps all old transitions and adds:

$$\delta(q_0, \varepsilon) = \{q_1, q_2\}.$$

Closure under Concatenation

Theorem: The class of regular languages is closed under the concatenation operation.

If A_1 and A_2 are regular, then $A_1 \circ A_2$ is regular.

Closure under Concatenation

Theorem: The class of regular languages is closed under the concatenation operation.

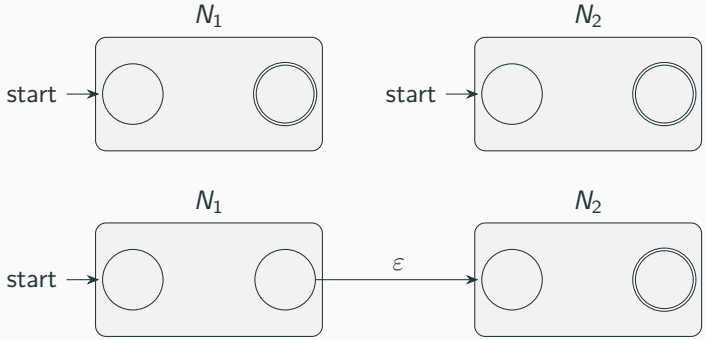
If A_1 and A_2 are regular, then $A_1 \circ A_2$ is regular.

Proof Idea: Given string w, we want to split it into $w = xy$ such that $x \in A_1$ and $y \in A_2$. But we don't know where the split point is!

NFA to the rescue: We guess the split point.

Proof: Closure under Concatenation

Let N_1 recognize A_1 and N_2 recognize A_2. We construct N to recognize $A_1 \circ A_2$.



Connect all accepting states of N_1 to the start state of N_2 using ε-transitions. The accept states of N_1 are no longer accepting.

Concatenation Construction: Formal Version

Let N_1 recognize A_1 and N_2 recognize A_2.

The new NFA starts in the start state of N_1 and accepts in the accepting states of N_2.

The key added transitions are:

$$\delta(r, \varepsilon) \ni q_2 \quad \text{for every } r \in F_1,$$

where q_2 is the start state of N_2.

Interpretation: after reading some prefix $x \in A_1$, the machine may jump into N_2 and try to read the remaining suffix $y \in A_2$.

Concatenation: Why Nondeterminism Is Natural

For A_1A_2, the machine must decide where the first part ends.

Example:

$$w = 001101$$

Maybe $w = 00\,1101$, or $w = 001\,101$, or $w = 0011\,01$.

A DFA can simulate all possibilities indirectly, but the NFA construction says the idea directly:

At an accepting state of N_1, guess that the split point is here.

Closure under Star

Theorem: The class of regular languages is closed under the star operation.

If A is regular, then A^* is regular.

Closure under Star

Theorem: The class of regular languages is closed under the star operation.

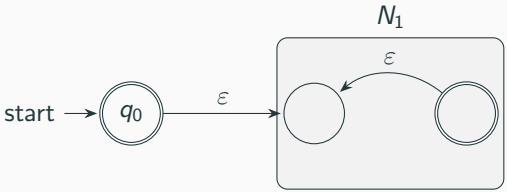
If A is regular, then A^* is regular.

Proof Idea: We have a machine N_1 for A. To accept A^*, the machine must accept sequences of strings from A.

When N_1 finishes reading one string in A, it should non-deterministically jump back to the beginning to start reading the next string.

Proof: Closure under Star

We must also accept ε, so we add a new accepting start state.



Add new start state q_0, an ε-arrow to the old start state, and ε-arrows from all old accept states back to the old start state.

Star Construction: Formal Version

Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A.

Create a new start state q_0 that is also accepting. Then:

$$Q = Q_1 \cup \{q_0\}, \qquad F = F_1 \cup \{q_0\}.$$

Add:

$$\delta(q_0, \varepsilon) \ni q_1$$

and for every $r \in F_1$:

$$\delta(r, \varepsilon) \ni q_1.$$

The new accepting start state accounts for the case of zero repetitions.

A Closure Proof Template

Most closure proofs have the same rhythm.

1. Start with machines for the input languages.
2. Describe the language operation we want to simulate.
3. Build a new finite automaton.
4. Argue both directions: every accepted string belongs to the target language, and every target string is accepted.

For regular operations, the construction is usually the heart of the proof.

Two Directions in a Correctness Proof

For a constructed machine N and target language L, we prove:

$$L(N) \subseteq L \quad \text{(machine accepts nothing extra)},$$
$$L \subseteq L(N) \quad \text{(machine accepts everything required)}.$$

Example for union: if N accepts w, some branch went into N_1 or N_2; if $w \in A_1 \cup A_2$, choose the branch for the accepting machine.

Because the class of regular languages is closed under Union, Concatenation, and Star, we can use these operations as a toolkit to build any regular language.

Union, Concatenation, and Star are the “Regular Operations”.

But regular languages are closed under many other operations too.

Closure under Complement

Definition: The complement of a language A, denoted $\overline{A}$, is the set of all strings in Σ^* that are **not** in A.

$$\overline{A} = \Sigma^* \setminus A$$

Theorem: The class of regular languages is closed under complement.

Closure under Complement

Definition: The complement of a language A, denoted $\overline{A}$, is the set of all strings in Σ^* that are **not** in A.

$$\overline{A} = \Sigma^* \setminus A$$

Theorem: The class of regular languages is closed under complement.

Proof Idea: If A is regular, there is a machine M that recognizes it. To build a machine that recognizes $\overline{A}$, we just run M and do the exact opposite of what it does.

If M accepts, we reject. If M rejects, we accept.

Proof: Closure under Complement

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a **DFA** recognizing A.

We construct a new DFA $M' = (Q, \Sigma, \delta, q_0, F')$ recognizing $\overline{A}$.

The only difference is the set of accepting states:

$$F' = Q \setminus F$$

Proof: Closure under Complement

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a **DFA** recognizing A.

We construct a new DFA $M' = (Q, \Sigma, \delta, q_0, F')$ recognizing $\overline{A}$.

The only difference is the set of accepting states:

$$F' = Q \setminus F$$

Warning! This construction **only works for DFAs**. If you flip the accept states of an NFA, the new NFA does not necessarily recognize the complement. (Why? Because an NFA accepts if *any* path reaches an accept state. Flipping states doesn't invert this logic correctly).

Complement Requires a Complete DFA

The complement construction assumes that every input string follows exactly one full computation path.

So before flipping accepting states, make sure the DFA is **complete**:

$$\delta(q, a) \text{ is defined for every } q \in Q \text{ and } a \in \Sigma.$$

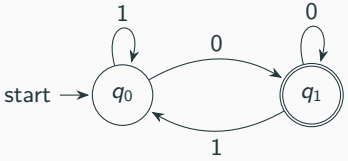
If a transition is missing, add a dead state and send the missing transition there.

This small detail prevents a common mistake: rejection by "getting stuck" is not the same as ending in a nonaccepting state unless the automaton model says so explicitly.

Example: Complementing a DFA

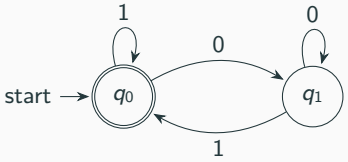
Language A: Strings ending in 0.

Original DFA (M):



Language $\overline{A}$: ε or strings ending in 1.

Complemented DFA (M'):



Closure under Intersection

Definition: $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$

Theorem: The class of regular languages is closed under intersection.

Closure under Intersection

Definition: $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$

Theorem: The class of regular languages is closed under intersection.

There are two beautiful ways to prove this:

1. **Constructive Proof (Product Construction):** Build a machine that runs M_1 and M_2 simultaneously.
2. **Algebraic Proof:** Use De Morgan's Laws.

Intersection Proof 1: Product Construction

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be DFAs.

We build $M = (Q, \Sigma, \delta, q_0, F)$ to simulate both. The states of M are pairs of states from M_1 and M_2.

- **States:** $Q = Q_1 \times Q_2 = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\}$
- **Start state:** $q_0 = (q_1, q_2)$
- **Transitions:** $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$

Intersection Proof 1: Product Construction

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be DFAs.

We build $M = (Q, \Sigma, \delta, q_0, F)$ to simulate both. The states of M are pairs of states from M_1 and M_2.

- **States:** $Q = Q_1 \times Q_2 = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\}$
- **Start state:** $q_0 = (q_1, q_2)$
- **Transitions:** $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$

To accept $A \cap B$, **both** machines must accept. Therefore:

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \textbf{ and } r_2 \in F_2\} = F_1 \times F_2$$

Reading Product States

In the product construction, a state (r, s) means:

After reading the same input prefix, M_1 is in state r and M_2 is in state s.

This is the finite-automata version of running two programs side by side.

For intersection, a product state is accepting only when both coordinates are accepting.

$$(r, s) \in F \quad \Longleftrightarrow \quad r \in F_1 \text{ and } s \in F_2.$$

Intersection Proof 2: De Morgan's Laws

A faster way to prove closure under intersection relies on what we already know.

Recall De Morgan's Law for sets:

$$A \cap B = \overline{\overline{A} \cup \overline{B}}$$

Intersection Proof 2: De Morgan's Laws

A faster way to prove closure under intersection relies on what we already know.

Recall De Morgan's Law for sets:

$$A \cap B = \overline{\overline{A} \cup \overline{B}}$$

We already proved that regular languages are closed under:

- **Complement** ($\overline{A}$ and $\overline{B}$ are regular)
- **Union** ($\overline{A} \cup \overline{B}$ is regular)
- **Complement again** ($\overline{\overline{A} \cup \overline{B}}$ is regular)

Therefore, $A \cap B$ must be regular. No new machines to build!

Closure under Difference

Definition: $A \setminus B = \{x \mid x \in A \text{ and } x \notin B\}$

Theorem: The class of regular languages is closed under set difference.

Closure under Difference

Definition: $A \setminus B = \{x \mid x \in A \text{ and } x \notin B\}$

Theorem: The class of regular languages is closed under set difference.

Proof: Notice that $A \setminus B$ is simply the intersection of A and the complement of B.

$$A \setminus B = A \cap \overline{B}$$

Since regular languages are closed under complement and intersection, they are closed under difference.

Closure under Reverse

Definition: The reverse of a string $w = c_1 c_2 \dots c_n$ is $w^R = c_n \dots c_2 c_1$. The reverse of a language A is $A^R = \{w^R \mid w \in A\}$.

Theorem: The class of regular languages is closed under reversal.

Closure under Reverse

Definition: The reverse of a string $w = c_1 c_2 \dots c_n$ is $w^R = c_n \dots c_2 c_1$. The reverse of a language A is $A^R = \{w^R \mid w \in A\}$.

Theorem: The class of regular languages is closed under reversal.

Proof Idea: If M is a machine that accepts a string by reading it forwards, we can build a new machine M^R that accepts by reading the string backwards.

To do this, we just reverse all the arrows in the transition diagram of M.

Building the Reverse Machine

Take a DFA M that recognizes A. We build an NFA N to recognize A^R.

1. Reverse the direction of every transition arrow.
2. The old start state becomes the new **single accept state**.
3. Create a **new start state**.
4. Draw ε-transitions from the new start state to all of the old accept states.

Building the Reverse Machine

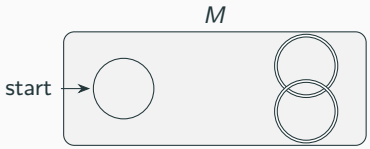
Take a DFA M that recognizes A. We build an NFA N to recognize A^R.

1. Reverse the direction of every transition arrow.
2. The old start state becomes the new **single accept state**.
3. Create a **new start state**.
4. Draw ε-transitions from the new start state to all of the old accept states.

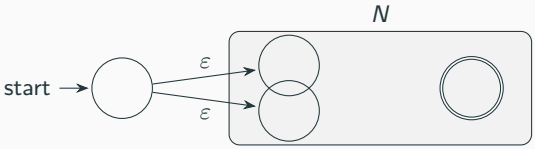
Why an NFA? Reversing arrows can turn a deterministic machine into a nondeterministic one (e.g., two arrows with the same symbol might now leave a state).

Visualizing Reversal

Original DFA for A:



Reversed NFA for A^R:



(Assume all internal arrows inside the box are reversed).

The Power of Closure Properties

By knowing these closure properties, we can prove languages are regular without ever drawing a state diagram!

Example: Let L be the language of strings over $\{0, 1\}$ that contain 101 but do **not** end in 11. Is L regular?

The Power of Closure Properties

By knowing these closure properties, we can prove languages are regular without ever drawing a state diagram!

Example: Let L be the language of strings over $\{0, 1\}$ that contain 101 but do **not** end in 11. Is L regular?

Yes.

- $L_1 = \{w \mid w \text{ contains } 101\}$ is regular (we built a DFA for it).
- $L_2 = \{w \mid w \text{ ends in } 11\}$ is regular.
- $L = L_1 \setminus L_2$.

Since regular languages are closed under set difference, L is regular.

Closure as a Proof Shortcut

Closure properties let us avoid building large automata manually.

Suppose A is the language of strings over $\{0, 1\}$ that contain 101, and B is the language of strings that have even length.

Then the language

$$A \cap B$$

is regular because both A and B are regular and regular languages are closed under intersection.

The proof is modular: prove small components regular, then combine them.

What is a Regular Expression?

We can use the regular operations to build up expressions describing languages, just as we use arithmetic operations $(+, \times)$ to build up expressions describing numbers.

Arithmetic Example: $(5 + 3) \times 4$ evaluates to 32.

Language Example: $(0 \cup 1)0^*$ evaluates to the language of all strings starting with a 0 or a 1, followed by any number of 0s.

What is a Regular Expression?

We can use the regular operations to build up expressions describing languages, just as we use arithmetic operations $(+, \times)$ to build up expressions describing numbers.

Arithmetic Example: $(5 + 3) \times 4$ evaluates to 32.

Language Example: $(0 \cup 1)0^*$ evaluates to the language of all strings starting with a 0 or a 1, followed by any number of 0s.

These expressions are called **Regular Expressions (RE)**.

Syntax Versus Semantics

A regular expression is a piece of syntax.

Its meaning is a language.

We often write $L(R)$ for the language described by the regular expression R.

Expression	**Syntax?**	**Language**
0	a symbol expression	$\{0\}$
0^*	star expression	$\{\varepsilon, 0, 00, 000, \dots\}$
$0 \cup 1$	union expression	$\{0, 1\}$

Formal Definition of a Regular Expression

We define regular expressions *inductively*.

R is a regular expression if R is:

1. a for some a in the alphabet Σ
2. ε (the empty string)
3. $\emptyset$ (the empty set)
4. $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions
5. $(R_1 \circ R_2)$, where R_1 and R_2 are regular expressions
6. (R_1^*), where R_1 is a regular expression

This tells us exactly what syntactical structures are valid REs.

The Semantic Definition $L(R)$

Sipser's formal definition is inductive. The language of R is defined recursively:

$$
\begin{aligned}
L(a) &= \{a\}, \\
L(\varepsilon) &= \{\varepsilon\}, \\
L(\emptyset) &= \emptyset, \\
L(R_1 \cup R_2) &= L(R_1) \cup L(R_2), \\
L(R_1 R_2) &= L(R_1) L(R_2), \\
L(R_1^*) &= L(R_1)^*.
\end{aligned}
$$

This is why induction is the natural proof technique for regular expressions.

Regular Expression Shorthands

The formal definition is intentionally small. In practice, we use shorthands.

Shorthand	Meaning
Σ	union of all alphabet symbols
R^+	RR^*
$R?$	$R \cup \varepsilon$
R^k	R concatenated with itself k times
[0-9]	$0 \cup 1 \cup \cdots \cup 9$

These are conveniences, not extra theoretical power.

Base Cases Explained

The first three rules are the base cases:

Expression R	**Language it describes, $L(R)$**
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\{\}$ (the empty set)

Notice the distinction: ε represents the language containing one string (which is empty). $\emptyset$ represents the language containing no strings at all.

Precedence of Operations

To avoid writing too many parentheses, we define an order of precedence for the regular operations, just like BEDMAS/PEMDAS in algebra.

Highest to lowest precedence:

1. Star (*)
2. Concatenation ($\circ$, usually omitted)
3. Union ($\cup$)

Example: $0 \cup 10^*$ means $0 \cup (1 \circ (0^*))$, **not** $(0 \cup 10)^*$ or $(0 \cup 1) \circ 0^*$.

Example 1

Let $\Sigma = \{0, 1\}$.

Expression: 0^*10^*

What language is this?

Example 1

Let $\Sigma = \{0, 1\}$.

Expression: 0^*10^*

What language is this?

Any number of 0s, followed by exactly one 1, followed by any number of 0s.

In English: $\{w \mid w \text{ contains exactly a single } 1\}$.

Example 2

Let $\Sigma = \{0, 1\}$.

Expression: $\Sigma^* 1 \Sigma^*$

(Note: Σ *is shorthand for* $(0 \cup 1)$*)*

What language is this?

Example 2

Let $\Sigma = \{0, 1\}$.

Expression: $\Sigma^* 1 \Sigma^*$

(Note: Σ *is shorthand for* $(0 \cup 1)$*)*

What language is this?

Any string, followed by a 1, followed by any string.

In English: $\{w \mid w \text{ contains at least one } 1\}$.

Example 3

Let $\Sigma = \{0, 1\}$.

Expression: $1^*\emptyset$

What language is this?

Example 3

Let $\Sigma = \{0, 1\}$.

Expression: $1^*\emptyset$

What language is this?

Concatenating any set with the empty set results in the empty set.

$L(1^*\emptyset) = \emptyset$.

Analogy: In arithmetic, multiplying anything by 0 *yields* 0.

Example 4

Let $\Sigma = \{0, 1\}$.

Expression: $\emptyset^*$

What language is this?

Example 4

Let $\Sigma = \{0, 1\}$.

Expression: $\emptyset^*$

What language is this?

By the definition of the Star operation, A^* always contains ε, even if A is empty. It concatenates zero elements of A.

$L(\emptyset^*) = \{\varepsilon\}$.

Example 5: At Least Two Ones

Let $\Sigma = \{0, 1\}$.

Language: all strings containing at least two 1s.

Example 5: At Least Two Ones

Let $\Sigma = \{0, 1\}$.

Language: all strings containing at least two 1s.

A clean expression is:

$$\Sigma^* 1 \Sigma^* 1 \Sigma^*.$$

The three Σ^* parts mean: anything before the first chosen 1, anything between two chosen 1s, and anything after the second chosen 1.

Translating English to Regular Expressions

Goal: Write a regex for the language over $\{0, 1\}$ where every string ends in 00.

Translating English to Regular Expressions

Goal: Write a regex for the language over $\{0, 1\}$ where every string ends in 00.

Answer: The string can start with anything, but must end with 00.

$$(0 \cup 1)^*00$$

Translating English to Regular Expressions

Goal: Write a regex for the language of strings of even length.

Translating English to Regular Expressions

Goal: Write a regex for the language of strings of even length.

Answer: Any symbol followed by any symbol gives a block of length 2. We want any number of these blocks.

$$((0 \cup 1)(0 \cup 1))^*$$

or simply

$$(\Sigma\Sigma)^*$$

Common Mistake: $\emptyset$ Versus ε

These two are not interchangeable.

Symbol	Language	Number of strings
$\emptyset$	no strings	0
ε	the empty string only	1

So:

$$R\emptyset = \emptyset, \qquad R\varepsilon = R.$$

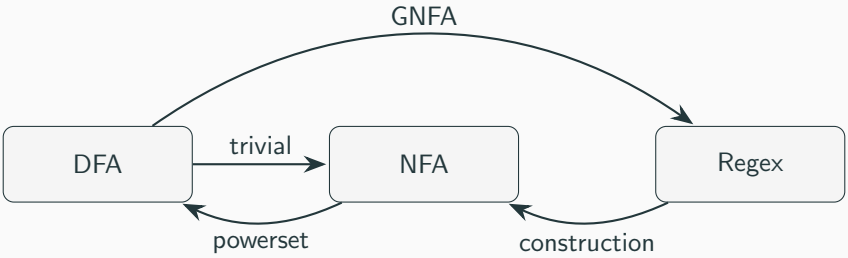
Useful Regex Identities

Just like in algebra, we can simplify regular expressions. For any regular expression R:

- $R \cup \emptyset = R$
- $R \circ \varepsilon = R$
- $R \cup R = R$
- $(R^*)^* = R^*$
- $R \circ \emptyset = \emptyset$

The Big Picture Theorem

We have three models on the table:



Kleene's Theorem completes the circle.

Kleene's Theorem

We now have two ways to describe languages:

1. Finite Automata (DFA/NFA)
2. Regular Expressions (RE)

Theorem (Kleene): A language is regular if and only if some regular expression describes it.

This implies two directions:

- **Lemma 1:** If a language is described by a regular expression, it is regular (can be converted to an NFA).
- **Lemma 2:** If a language is regular (recognized by a DFA), it can be described by a regular expression.

Proof of Lemma 1: RE $\rightarrow$ NFA

We prove this using structural induction on the definition of a regular expression.

We first build NFAs for the base cases: $R = a$, $R = \varepsilon$, and $R = \emptyset$.

Then we show how to combine them for the inductive steps: Union, Concatenation, and Star.
(Wait, we already proved the closure properties! We will reuse those constructions.)

Why Structural Induction Works Here

A regular expression is built from smaller regular expressions.

To prove every regular expression can be converted to an NFA:

1. Handle the atomic expressions: a, ε, and $\emptyset$.
2. Assume R_1 and R_2 already have NFAs.
3. Build NFAs for $R_1 \cup R_2$, $R_1 R_2$, and R_1^*.

This follows the grammar of regular expressions exactly.

RE to NFA: Base Cases

Case 1: $R = a$

Case 2: $R = \varepsilon$

Case 3: $R = \emptyset$

RE to NFA: Inductive Steps

If $R = R_1 \cup R_2$, $R = R_1 \circ R_2$, or $R = R_1^*$, we assume we already have NFAs N_1 and N_2 for R_1 and R_2.

We simply plug N_1 and N_2 into the constructions we proved earlier during the Closure Properties section!

This systematic translation is often called **Thompson's Construction**.

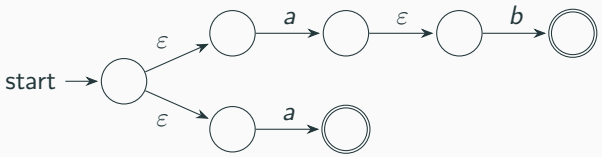
Example: Convert $(ab \cup a)^*$ to NFA

Step 1: Build NFAs for a and b.

Step 2: Concatenate them to get ab.

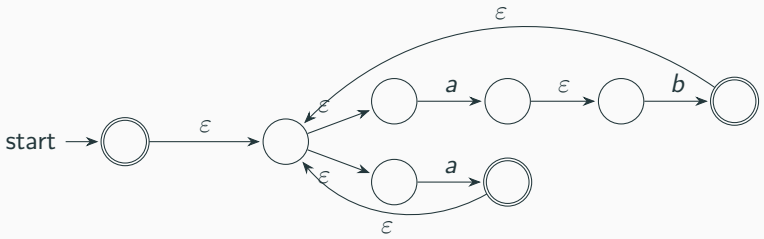
Example: Convert $(ab \cup a)^*$ to NFA

Step 3: Union with NFA for a to get $(ab \cup a)$.



Example: Convert $(ab \cup a)^*$ to NFA

Step 4: Apply Star to the whole machine.



We can easily construct an NFA for any RE.

Proof of Lemma 2: DFA $\rightarrow$ RE

If a language is regular, it is recognized by a DFA. We need an algorithm to convert a DFA into a Regular Expression.

We will do this using a new type of machine called a **Generalized Nondeterministic Finite Automaton (GNFA)**.

A GNFA is an NFA where the transitions are labeled by **entire regular expressions**, not just single symbols or ε.

What is a GNFA?

In a GNFA, a transition from q_i to q_j labeled with R means: The machine can travel from q_i to q_j if it reads a block of input that matches the regular expression R.

GNFA Transition Function

A GNFA has a transition function of the form

$$\delta : (Q \setminus \{q_{accept}\}) \times (Q \setminus \{q_{start}\}) \to \mathcal{R},$$

where $\mathcal{R}$ is the set of all regular expressions over Σ.

So each allowed ordered pair of states has exactly one regular-expression label.

If there is no real way to move from p to q, the label is $\emptyset$.

The Rules of a Sipser GNFA

For our conversion algorithm, we enforce a strict form on the GNFA:

1. The start state has transitions going *to* every other state, but no transitions coming *in*.
2. There is a single accept state, which has transitions coming *in* from every other state, but no transitions going *out*.
3. The start state and accept state are different.
4. Except for the start and accept states, one arrow goes from every state to every other state, and also from each state to itself.

Preparing a DFA for the GNFA Algorithm

Given a DFA, how do we force it into this strict GNFA form?

1. Add a **new start state** with an ε-transition to the old start state.
2. Add a **new single accept state** with ε-transitions from all old accept states.
3. If any required transitions are missing, add them with the label $\emptyset$.
4. If a state has multiple parallel transitions to another state (e.g., on a and on b), combine them using Union ($a \cup b$).

Why Add New Start and Accept States?

The new start state makes sure no transition enters the start.

The new accept state makes sure no transition leaves the accept state and there is only one final state.

These rules simplify the algorithm: at the end, exactly two states remain and the one remaining label is the answer.

Without this normalization, the final expression would require extra case handling.

The Core Idea: State Ripping

Once we have a GNFA, we will successively remove ("rip") internal states one by one, while preserving the language recognized.

When we rip a state q_{rip}, we must repair the transitions between all remaining pairs of states q_i and q_j.

Eventually, only the start state and the accept state will remain. The single transition between them will be the final Regular Expression!

The Ripping Formula

Suppose we want to rip state q_{rip}. For any pair of remaining states q_i and q_j, we update the direct transition label $R(q_i, q_j)$.

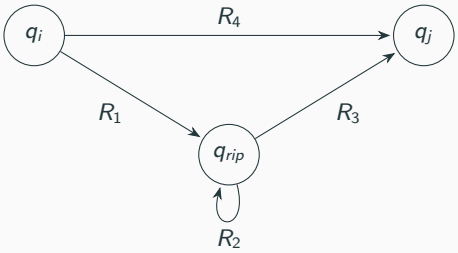
The old machine could go from q_i to q_j directly, OR it could go through q_{rip}.

The new label must be:

$$R_{new}(q_i, q_j) = R_{old}(q_i, q_j) \cup (R(q_i, q_{rip}) \circ (R(q_{rip}, q_{rip}))^* \circ R(q_{rip}, q_j))$$

Visualizing the State Rip

Before ripping q_{rip}:



After ripping q_{rip}:

Different Regexes Can Describe the Same Language

Regular expressions are not unique.

For strings ending in 1, all of the following describe the same language:

$$(0 \cup 1)^* 1,$$

$$0^* 1 (1 \cup 00^* 1)^*,$$

$$\Sigma^* 1.$$

The GNFA algorithm gives a correct expression, not necessarily the prettiest expression.

Summary of Kleene's Theorem

We have shown a complete lifecycle:

1. **Regular Expressions** precisely define patterns algebraically.
2. We can convert any RE into an **NFA** (using base structures and closure properties).
3. Every NFA is equivalent to a **DFA** (using powerset construction).
4. We can convert any DFA back into a **Regular Expression** (using GNFA and state ripping).

Summary of Kleene's Theorem

We have shown a complete lifecycle:

1. **Regular Expressions** precisely define patterns algebraically.
2. We can convert any RE into an **NFA** (using base structures and closure properties).
3. Every NFA is equivalent to a **DFA** (using powerset construction).
4. We can convert any DFA back into a **Regular Expression** (using GNFA and state ripping).

Therefore, DFAs, NFAs, and Regular Expressions all describe exactly the same class of languages: **The Regular Languages**.